

NFV Project

Viva Questions & Answers

Counter Question Preparation Checklist

Diksha

May 16, 2026

Contents

1 Docker	3
2 Kubernetes	3
3 Ingress (Deep Dive)	4
4 Jenkins & CI/CD	5
5 Prometheus & Grafana (Monitoring)	6
6 Elasticsearch & Kibana (Logging)	6
7 Monitoring vs Logging	7
8 Flask & Python	8
9 Networking Concepts	8
10 Microservices Architecture	8
11 NFV / Project-Specific Questions	9
12 Git & Version Control	10
13 DevOps Concepts	10
14 Quick Revision Cheatsheet	10

1 Docker

Q1: What is Docker?

Docker is a platform that packages applications into containers — lightweight, standalone units that include everything needed to run the software (code, runtime, libraries, dependencies). It ensures the app runs the same everywhere regardless of the host environment.

Q2: What is the difference between a Docker Image and a Container?

An **image** is a read-only template (like a class in OOP). A **container** is a running instance of that image (like an object). You can run multiple containers from the same image.

Q3: What is a Dockerfile?

A text file with instructions to build a Docker image. Each instruction creates a layer. Example instructions: `FROM` (base image), `COPY` (add files), `RUN` (execute commands), `EXPOSE` (declare port), `CMD` (default command).

Q4: What is Docker Compose?

A tool for defining and running multi-container applications using a YAML file (`docker-compose.yml`). Instead of running multiple `docker run` commands, you define all services in one file and start them with `docker-compose up`.

Q5: What is the difference between CMD and ENTRYPOINT?

`CMD` provides default arguments that can be overridden at runtime. `ENTRYPOINT` sets the main command that always runs — `CMD` becomes arguments to it.

Q6: What is a Docker volume?

A mechanism to persist data outside the container's filesystem. When a container is deleted, its data is lost — volumes survive container restarts and deletions.

Q7: What is DockerHub?

A cloud registry to store and share Docker images (like GitHub for code). You push images with `docker push` and pull with `docker pull`.

Q8: What does `docker build -t dknight/firewall:v1 .` do?

Builds a Docker image from the Dockerfile in the current directory (`.`), tags it with the name `dknight/firewall` and version `v1`.

2 Kubernetes

Q1: What is Kubernetes?

An open-source container orchestration platform that automates deployment, scaling, and management of containerized applications. It handles load balancing, self-healing, rolling updates, and service discovery.

Q2: What is Minikube?

A tool that runs a single-node Kubernetes cluster locally for development and testing. It creates a VM or container that acts as both master and worker node.

Q3: What is a Pod?

The smallest deployable unit in Kubernetes. A pod is a wrapper around one or more containers that share the same network namespace and storage. Usually one container per pod.

Q4: What is a Deployment?

A Kubernetes resource that manages pods. It ensures the desired number of pod replicas are running, handles rolling updates, and provides self-healing (restarts crashed pods).

Q5: What is a Service in Kubernetes?

An abstraction that provides a stable network endpoint (IP + DNS name) to access pods. Pods are ephemeral (they get new IPs on restart), but services provide a fixed address.

Q6: What is ClusterIP vs NodePort vs LoadBalancer?

- **ClusterIP** — Only accessible inside the cluster (default). Used for internal communication between services.
- **NodePort** — Exposes the service on a fixed port on every node's IP. Accessible from outside the cluster.
- **LoadBalancer** — Creates an external load balancer (cloud only). Routes external traffic to the service.

Q7: What is an Ingress?

A Kubernetes resource that manages external HTTP/HTTPS access to services. It acts as a reverse proxy — routes traffic based on hostname or URL path to the appropriate service. It requires an Ingress Controller (like NGINX) to function.

Q8: What is the difference between Ingress and NodePort?

NodePort exposes a service on a random high port (30000-32767) on every node. Ingress provides proper HTTP routing with hostnames and paths on port 80/443 — much cleaner for web traffic.

Q9: What is a Namespace?

A logical isolation boundary within a cluster. Resources in different namespaces are separated. Used to organize resources by project, team, or environment (dev/staging/prod).

Q10: What is a ConfigMap?

A Kubernetes object that stores non-sensitive configuration data as key-value pairs. Pods can consume it as environment variables or mounted files. In our project, Prometheus scrape config is stored as a ConfigMap.

Q11: What is a PersistentVolume (PV) and PersistentVolumeClaim (PVC)?

PV is a piece of storage provisioned by an admin. **PVC** is a request for storage by a pod. The PVC binds to a PV. This ensures data survives pod restarts. In our project, Elasticsearch uses a PVC for log persistence.

Q12: What does `kubectl apply -f` do?

Applies the configuration defined in a YAML file to the cluster. It creates or updates the resource. It's declarative — you describe the desired state and Kubernetes makes it happen.

Q13: What is `kubectl rollout restart`?

Forces a deployment to restart all its pods (creates new ones, terminates old ones). Used to pull new Docker images or reset pod state without changing the manifest.

Q14: What is self-healing in Kubernetes?

If a pod crashes or a node fails, Kubernetes automatically restarts the pod or reschedules it to a healthy node to maintain the desired number of replicas.

3 Ingress (Deep Dive)

Q1: What is an Ingress Controller?

A pod running a reverse proxy (usually NGINX) that watches Ingress resources and configures routing rules. Without an Ingress Controller, Ingress resources do nothing.

Q2: How does Ingress work in your project?

We have an NGINX Ingress Controller (enabled via `minikube addons enable ingress`).

Our Ingress resource maps hostname `nfv.local` to the Switch service on port 5001. When a request hits `http://nfv.local/route`, the Ingress Controller routes it to the Switch pod.

Q3: Why use Ingress instead of directly exposing services?

- Single entry point for all traffic
- Host-based and path-based routing
- SSL/TLS termination in one place
- No need to remember random NodePort numbers

Q4: What does `host: nfv.local` mean in Ingress?

It tells the Ingress Controller to only route requests that have `Host: nfv.local` in their HTTP header to this service. That's why we add it to `/etc/hosts`.

4 Jenkins & CI/CD

Q1: What is CI/CD?

- **CI (Continuous Integration)** — Automatically build and test code on every commit
- **CD (Continuous Delivery/Deployment)** — Automatically deploy tested code to production

Q2: What is Jenkins?

An open-source automation server that runs CI/CD pipelines. It executes a sequence of stages (build, test, deploy) triggered by code changes or manual runs.

Q3: What is a Jenkinsfile?

A file that defines the pipeline as code. It lives in the repo alongside the application code. Uses Groovy-based DSL with stages, steps, and post actions.

Q4: What is a Jenkins Pipeline?

A series of automated steps that take code from source to deployment. Stages run sequentially. If any stage fails, the pipeline stops.

Q5: What is `withCredentials` in Jenkins?

A step that securely injects stored credentials into the pipeline without exposing them in logs. In our project, it injects DockerHub username/password for `docker login`.

Q6: What is the difference between Declarative and Scripted Pipeline?

Declarative (what we use) has a fixed structure with `pipeline {}`, `stages {}`, `steps {}`. **Scripted** is freeform Groovy code with `node {}`. Declarative is simpler and recommended.

Q7: What does `agent any` mean?

Run the pipeline on any available Jenkins agent (executor). If you have multiple build machines, Jenkins picks whichever is free.

Q8: What happens if the pipeline fails at Push stage?

The Deploy stage won't run. Jenkins marks the build as failed. The `post { failure {} }` block executes. Images are built locally but not pushed to DockerHub and not deployed.

Q9: What is a webhook in Jenkins?

A mechanism where GitHub notifies Jenkins immediately when code is pushed, triggering the pipeline automatically instead of polling.

5 Prometheus & Grafana (Monitoring)

Q1: What is Prometheus?

An open-source time-series database and monitoring system. It **pulls** (scrapes) metrics from target applications at regular intervals and stores them with timestamps.

Q2: What is a metric?

A numerical measurement tracked over time. Examples: request count, error rate, CPU usage, response time. In our project: `allowed_requests_total` and `blocked_requests_total`.

Q3: What is a Counter vs Gauge vs Histogram?

- **Counter** — Only goes up (total requests, total errors). We use this.
- **Gauge** — Goes up and down (CPU usage, temperature, active connections)
- **Histogram** — Tracks distribution of values (request duration buckets)

Q4: What does scraping mean?

Prometheus periodically sends HTTP GET requests to the `/metrics` endpoint of target services. It parses the response and stores the values. In our project, it scrapes `firewall:5000/metrics` every 5 seconds.

Q5: What is Grafana?

A visualization platform that connects to data sources (like Prometheus) and displays metrics as dashboards with graphs, gauges, pie charts, and alerts.

Q6: Why use Grafana if Prometheus has a UI?

Prometheus UI is for ad-hoc queries and debugging. Grafana provides persistent dashboards, auto-refresh, alerting, multiple panels, and a much better visual experience for ongoing monitoring.

Q7: What is PromQL?

Prometheus Query Language. Used to query metrics. Examples:

- `allowed_requests_total` — current value
- `rate(allowed_requests_total[1m])` — requests per second over last minute
- `blocked_requests_total / allowed_requests_total` — ratio

Q8: What is a scrape interval?

How often Prometheus fetches metrics from targets. We set it to 5 seconds. Lower = more granular data but more storage/network usage.

6 Elasticsearch & Kibana (Logging)

Q1: What is Elasticsearch?

A distributed search and analytics engine. It stores JSON documents and lets you search through them extremely fast using inverted indexes. Think of it as a database optimized for search.

Q2: What is the difference between Elasticsearch and a regular database?

Regular databases (MySQL, PostgreSQL) are optimized for structured data and transactions. Elasticsearch is optimized for full-text search, filtering, and analytics on semi-structured data like logs.

Q3: What is Kibana?

A web UI for Elasticsearch. It lets you search, filter, and visualize data stored in Elasticsearch. You can build dashboards, create alerts, and explore logs interactively.

Q4: What is an Index in Elasticsearch?

Similar to a database table. It's a collection of documents with similar characteristics. In our project, the index is called `logs` — all traffic logs go there.

Q5: What is a Document in Elasticsearch?

A single JSON record stored in an index. In our project, each document is:

```
{
  "ip": "192.168.49.1",
  "data": {"message": "hello"},
  "timestamp": "2026-05-15T15:39:51"
}
```

Q6: Why Elasticsearch instead of just storing logs in a file?

- Searchable — find specific logs instantly among millions
- Scalable — distributes data across multiple nodes
- Structured — JSON format allows filtering by any field
- Visualization — Kibana provides dashboards without custom code

Q7: What is the ELK stack?

Elasticsearch (storage) + Logstash (processing/parsing) + Kibana (visualization). We use E+K directly without Logstash since our Monitor service formats logs before sending.

Q8: How do logs get into Elasticsearch in your project?

Monitor service receives traffic data from Switch → creates a JSON entry → sends HTTP POST to `http://elasticsearch:9200/logs/_doc` → Elasticsearch indexes it → Kibana can search it.

7 Monitoring vs Logging

Q1: What is the difference between monitoring and logging?**Monitoring (Prometheus + Grafana)**

- Tracks NUMBERS over time (metrics)
- Answers: “How many? How fast? How often?”
- Example: 50 requests blocked in the last hour
- Good for: Dashboards, alerts, trends

Logging (Elasticsearch + Kibana)

- Stores FULL RECORDS (logs)
- Answers: “What happened? Who did it? When exactly?”
- Example: IP 10.0.0.1 sent “DROP table” at 3:42:17 PM
- Good for: Debugging, forensics, auditing

Q2: Why do you need both?

Monitoring tells you *something is wrong* (spike in blocked requests). Logging tells you *what went wrong* (which IPs, which payloads). You need monitoring for real-time awareness and logging for investigation.

8 Flask & Python

Q1: What is Flask?

A lightweight Python web framework for building APIs and web applications. It provides routing (`@app.route`), request handling, and JSON responses with minimal boilerplate.

Q2: Why Flask for this project?

It's minimal and fast to set up. Each microservice only needs a few endpoints. Flask's simplicity makes it ideal for lightweight services that do one thing well.

Q3: What is `request.json` in Flask?

It parses the incoming HTTP request body as JSON and returns a Python dictionary. Only works when `Content-Type: application/json` header is sent.

Q4: What does `app.run(host="0.0.0.0")` mean?

Listens on all network interfaces (not just localhost). This is required inside Docker containers so external traffic can reach the app.

Q5: What is `prometheus_client` library?

A Python library that exposes application metrics in Prometheus format. You create Counter/Gauge objects, increment them in your code, and expose them at `/metrics` endpoint.

9 Networking Concepts

Q1: What is a port?

A number (0-65535) that identifies a specific process on a machine. Port 80 = HTTP, Port 443 = HTTPS, Port 5000 = our firewall service.

Q2: What is DNS?

Domain Name System — translates hostnames to IP addresses. Inside Kubernetes, services get DNS names automatically (e.g., `firewall` resolves to the service's ClusterIP).

Q3: What is a reverse proxy?

A server that sits in front of backend services and forwards client requests to them. The Ingress Controller (NGINX) is a reverse proxy — clients talk to it, and it routes to the correct service.

Q4: What does `X-Forwarded-For` header do?

When a request passes through proxies, the original client IP is lost. `X-Forwarded-For` preserves the original IP. Our firewall checks this header to identify the real source.

Q5: What is ClusterIP in Kubernetes networking?

A virtual IP assigned to a service that's only reachable within the cluster. Other pods can reach it by service name (DNS). External traffic cannot access ClusterIP services directly.

10 Microservices Architecture

Q1: What are microservices?

An architecture where an application is split into small, independent services that communicate over the network. Each service does one thing, can be deployed independently, and can use different technologies.

Q2: Why microservices instead of monolith?

- Independent scaling (scale only what needs it)

- Independent deployment (deploy firewall without restarting switch)
- Technology freedom (each service can use different language)
- Fault isolation (one service crashing doesn't take down everything)

Q3: How do your microservices communicate?

Via HTTP REST APIs over the internal Kubernetes network. Switch calls Firewall at `http://firewall:5000/check` and Monitor at `http://monitor:5002/log`. Kubernetes DNS resolves service names to IPs.

Q4: What happens if the Monitor service goes down?

The Switch still works. It sends requests to Firewall and returns the response. The Monitor call might fail (timeout), but it doesn't block the main flow since it's for logging only.

Q5: What is service discovery?

The mechanism by which services find each other. In Kubernetes, this is automatic via DNS — service names resolve to their ClusterIP. No hardcoded IPs needed.

11 NFV / Project-Specific Questions

Q1: What is Network Function Virtualization?

Replacing dedicated network hardware (physical firewalls, routers, load balancers) with software running on commodity servers. Our project virtualizes a firewall, switch, and monitor as containerized microservices.

Q2: Why virtualize network functions?

- Cost reduction (no expensive proprietary hardware)
- Flexibility (deploy/update instantly)
- Scalability (spin up more instances on demand)
- Automation (CI/CD for network infrastructure)

Q3: How does your firewall decide what to block?

Two rules: (1) Check if source IP is in the blocklist (192.168.1.10, 10.0.0.1). (2) Check if request content contains malicious keywords (DROP, malicious, attack). If either matches, return 403 Blocked.

Q4: Why does the Switch send to both Firewall and Monitor?

Security (Firewall) and observability (Monitor) are separate concerns. The Monitor logs **all** traffic — including blocked requests — for auditing purposes. You need to know what attacks were attempted, not just what was allowed.

Q5: What metrics does Prometheus collect from your firewall?

- `allowed_requests_total` — Counter of requests that passed the firewall
- `blocked_requests_total` — Counter of requests that were blocked

Q6: What would you add to make this production-ready?

- Rate limiting on the firewall
- Authentication/authorization (JWT tokens)
- HTTPS/TLS encryption
- Horizontal pod autoscaling
- Alerting rules in Prometheus (alert on spike in blocked requests)
- Log rotation and retention policies in Elasticsearch

12 Git & Version Control

Q1: What is Git?

A distributed version control system that tracks changes to files. Every developer has a full copy of the repository history locally.

Q2: What is the difference between Git and GitHub?

Git is the version control tool (runs locally). GitHub is a cloud platform that hosts Git repositories and adds collaboration features (pull requests, issues, actions).

Q3: What is a branch?

A separate line of development. You work on features in branches and merge them into `main` when ready. This avoids breaking the main codebase.

Q4: What is `git push`?

Uploads local commits to the remote repository (GitHub). Makes your changes available to others and to Jenkins for CI/CD.

13 DevOps Concepts

Q1: What is DevOps?

A culture and set of practices that combines software development (Dev) and IT operations (Ops). Goal: faster, more reliable software delivery through automation, CI/CD, monitoring, and collaboration.

Q2: What is Infrastructure as Code (IaC)?

Managing infrastructure through code files instead of manual configuration. Our Kubernetes YAML files are IaC — the entire deployment is defined in version-controlled files.

Q3: What is a rolling update?

Kubernetes replaces old pods with new ones one at a time (not all at once). This ensures zero downtime — some pods always remain running to serve traffic during the update.

Q4: What is the difference between `docker-compose` and Kubernetes?

`docker-compose` is for local development (single machine, simple orchestration). Kubernetes is for production (multi-node, auto-scaling, self-healing, load balancing, rolling updates).

Q5: What is container orchestration?

Automating the deployment, scaling, networking, and management of containers across multiple machines. Kubernetes is the most popular container orchestrator.

Q6: What is a container registry?

A storage service for Docker images. DockerHub is a public registry. You push images after building and pull them during deployment.

14 Quick Revision Cheatsheet

Term	One-line Definition
Docker	Packages apps into portable containers
Dockerfile	Instructions to build a Docker image
Kubernetes	Orchestrates containers at scale
Pod	Smallest unit in K8s (wrapper around container)

Deployment	Manages pod replicas and updates
Service	Stable network endpoint for pods
Ingress	Routes external HTTP to internal services
Namespace	Logical isolation within a cluster
ConfigMap	Stores configuration as K8s resource
PersistentVolume	Durable storage that survives pod restart
Minikube	Local single-node K8s cluster
Jenkins	CI/CD automation server
Pipeline	Automated build-test-deploy workflow
Prometheus	Scrapes and stores metrics (time-series)
Grafana	Visualizes metrics as dashboards
Elasticsearch	Stores and searches JSON documents (logs)
Kibana	UI to explore Elasticsearch data
Flask	Python web framework for APIs
Microservices	App split into small independent services
CI/CD	Automate build, test, and deployment
NFV	Replace hardware network devices with software
Reverse Proxy	Routes client requests to backend services
Rolling Update	Zero-downtime deployment strategy
IaC	Manage infrastructure through code files